챕터 3. 세션을 바깥에 둬라. Redis 세션 공유

이번 챕터가 끝나면

- 서버 메모리에 세션을 올리면 왜 2대 이상에서 무너지는지 직접 확인합니다 (세션 불일치)
- 세션을 서버 바깥의 공용 저장소로 옮기는 구조를 익힙니다 (Spring Session Data Redis)
- 같은 브라우저가 app1과 app2를 오가도 로그인 상태가 유지되는 걸 눈으로 확인합니다 (세션 공유 검증)

이번 챕터는 "서버가 두 대가 되는 순간"에 대한 이야기입니다

챕터 2에서 카카오 로그인을 붙여 세션에 사용자를 넣어 두었습니다. 그런데 이 세션은 지금 컨테이너 한 대의 메모리 안에만 있습니다. 트래픽이 늘어 컨테이너를 한 대 더 띄우는 순간, 옆 컨테이너는 세션을 모릅니다. 로그인한 사용자가 새로고침 한 번에 로그아웃되는 경험이 시작됩니다. 이 챕터에서는 그 문제를 그대로 재현한 뒤, 세션을 **컨테이너 바깥의 Redis**로 옮겨서 모든 컨테이너가 같은 세션을 바라보도록 만들겠습니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 3의 실습 레포는 하나입니다. 인프라와 두 개의 Spring 서버가 한 `docker-compose.yml`로 묶여 있습니다.

레포	용도	주소
docker-session-share	Nginx + Spring app1 + Spring app2 + Redis 4단 구성	`github.com/metacoding-11-spring-reference/docker-session-share`

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/docker-session-share.git
cd docker-session-share
```

파일 구조는 이렇습니다.

docker-session-share 디렉토리

```
docker-session-share/
├── docker-compose.yml              # [실습] 네 컨테이너(Nginx·app1·app2·Redis) 배치도
├── nginx/
│   └── nginx.conf                  # [실습] Path 기반 라우팅 설정
├── app1/                           # [참고] Spring Boot 서버 A (/app1)
│   ├── build.gradle
│   ├── src/main/resources/application.properties
│   └── src/main/java/.../HomeController.java
└── app2/                           # [참고] Spring Boot 서버 B (/app2)
    └── (app1과 동일 구조)
```

> **두 Spring 서버의 코드는 동일합니다.** 응답에 찍히는 서버 이름(`app1` / `app2`)만 다르고, 세션을 읽고 쓰는 로직은 똑같습니다. 두 대를 띄우는 것이 목적이기 때문입니다.

2. 실습 환경 구축

챕터 1에서 깔아 둔 **Docker Desktop** 하나면 충분합니다. 새로 깔 것은 없습니다. 다만 이번부터 `docker compose up`이 띄우는 컨테이너 수가 늘어나므로, Docker Desktop의 **Resources** 설정에서 메모리가 4GB 이상 할당되어 있는지 한 번 확인해 주세요.

터미널 실습 환경 확인

```
docker --version
docker compose version
```

3. 사용할 구성 요소

챕터 3에 새로 얹히는 재료는 세 가지입니다.

재료	역할
`redis:7-alpine` 이미지	메모리 기반 키-값 저장소. 세션이 머무는 바깥 창고
`spring-session-data-redis`	Spring의 `HttpSession`을 Redis에 자동으로 저장해 주는 모듈
`nginx:alpine` 이미지	같은 `/` 아래로 들어온 요청을 app1과 app2에 나눠주는 라우터

Spring Session을 쓰는 이유

`HttpSession`을 직접 Redis에 직렬화해서 넣는 코드를 손으로 짤 수도 있습니다. 하지만 그러면 로그인·장바구니·인증 필터가 각자 다른 키에 저장되고, 세션 생성·만료·재발급을 전부 직접 관리해야 합니다. `spring-session-data-redis`는 이 모든 걸 `HttpSession` 인터페이스 뒤에 가려 줍니다. 우리는 `session.setAttribute()`를 그대로 쓰되, 실제 저장소만 메모리에서 Redis로 바뀝니다.

4. 실습 순서

이번 챕터의 절은 이 순서로 진행합니다.

1. `3.1`. 컨테이너 한 대 환경의 한계 체감
2. `3.2`. 인메모리 세션이 왜 두 대에서 무너지는지 직접 확인
3. `3.3`. 세션을 바깥으로 꺼내는 비유와 구조 잡기
4. `3.4`. Spring Session Data Redis 설정하기 (build.gradle + application.properties)
5. `3.5`. Nginx 라우터와 docker-compose 네 컨테이너 구성
6. `3.6`. 같은 브라우저로 `/app1 → /app2 → /app1` 왕복하며 세션 공유 확인
7. `3.7`. Redis CLI로 세션 키 구조와 TTL 직접 들여다보기

1~2에서 문제를 만난 뒤 3에서 비유로 길을 잡습니다. 4~5에서 Redis와 Nginx를 얹고, 6에서 눈으로 확인한 뒤, 7에서 저장소 안을 들여다보겠습니다.

3.1 한 대일 때는 몰랐던 것

카카오 로그인을 붙인 날의 오후.

형광등이 조용히 깜빡였습니다. 오픈이는 `/post/list` 페이지를 새로고침했습니다. 닉네임이 상단에 떠 있었습니다. 잘 돌아가네. 의자를 한 번 돌렸다가 다시 모니터 쪽으로 돌아왔습니다.

옆자리 동료가 먼저 말을 걸었습니다.

동료 : "저도 로그인해 봤어요. 그런데 좀 이상해요."

오픈이 : "어디가요?"

동료 : "방금 로그인했는데, 새로고침 한 번 했더니 로그아웃돼 있어요. 그래서 다시 로그인했더니, 또 다음 요청에선 풀려 있네요."

그럴 리가.

오픈이는 화면을 봤습니다. 자기 쪽에서는 새로고침해도 로그인이 유지되고 있었습니다. 뭔가 다른 점이 있어야 했습니다. 동료 자리로 가 봤습니다.

동료 : "저는 이렇게 했어요. 오픈이 씨가 띄운 서버 말고, 제가 따로 `docker compose up` 해서 8081 포트로 하나 더 띄웠거든요. 그래서 저는 8081로 접속해요."

두 대.

그게 문제였습니다. 정확히 말하면, 둘 다 잘 돌아가고 있었는데 **두 대가 있다는 사실 자체**가 문제였습니다.

오픈이는 자기 자리로 돌아와 수첩을 폈습니다. 카카오 로그인 흐름을 다시 머리에 그렸습니다. 사용자가 `/login/kakao`를 누른다. 카카오로 튀어갔다가 `code`를 들고 돌아온다. 오픈이의 서버가 code를 토큰으로 바꾸고 사용자 정보를 뽑는다. 그리고 마지막엔 세션에 사용자 객체를 꽂는다.

그런데 여기서 세션이 어디 있더라.

자바의 `HttpSession`은 Tomcat이 관리합니다. Tomcat은 이 서버 프로세스 안에 떠 있습니다. 그러니까 세션은 **이 컨테이너의 메모리 안**에 있었습니다.

아.

동료가 로그인한 건 **8081 컨테이너의 메모리**였습니다. 그 다음 새로고침 요청이 어디로 날아갔을까요. 브라우저는 `localhost:8081` 그대로 요청을 보냈을 겁니다. 똑같은 컨테이너였으니 세션이 남아 있어야 맞는데, 동료 말로는 풀려 있다고 했습니다. 다시 돌아가서 물어봤습니다.

오픈이 : "8081 말고 다른 포트로도 띄우셨어요?"

동료 : "아뇨. 근데 조금 전에 `docker compose restart`를 한번 했어요. 느려져서요."

다시 켠 거구나.

컨테이너를 재시작하면 메모리가 날아갑니다. 그러니까 세션도 같이 날아갔습니다. 동료가 로그인한 그 세션은 이전 컨테이너 프로세스의 힙 어딘가에 있었는데, 재시작한 컨테이너는 전혀 다른 프로세스였습니다. 서로의 메모리를 공유하지 않았습니다.

이건 그나마 약한 증상이었습니다. 팀장이 점심 먹고 돌아오다가 지나가는 말로 한마디 던졌습니다.

팀장 : "다음 주에 트래픽 테스트해 볼 거예요. 오픈이 씨, 그때는 Spring 컨테이너 두 개 띄워 보세요. 앞에 Nginx 하나 두면 되겠네요."

두 개.

컨테이너 하나를 더 띄우는 순간 지금의 증상은 훨씬 심해질 예정이었습니다. 사용자가 `/login/kakao`로 요청을 보냈을 때 Nginx가 **왼쪽 컨테이너**로 보낼 수도 있고, **오른쪽 컨테이너**로 보낼 수도 있었습니다. 어느 쪽이 잡느냐는 그때그때 다릅니다. 로그인해서 세션을 만든 게 왼쪽 컨테이너라면, 그 다음 요청이 오른쪽으로 가는 순간 오른쪽은 아무것도 모릅니다. 누구세요?

팀장 : "포스트잇을 네 책상 서랍에 넣어 두면, 옆자리 사람이 그걸 찾을 수가 없어요. 생각해봐요."

팀장은 자리로 돌아갔습니다.

책상 서랍.

서랍이 세션 저장소였습니다. 왼쪽 컨테이너의 서랍, 오른쪽 컨테이너의 서랍이 따로였습니다. 컨테이너가 재시작되면 서랍째 통째로 날아갔습니다. 그럼 포스트잇을 어디에 붙여야 하지.

오픈이는 수첩 귀퉁이에 낙서를 했습니다.

— 생각 정리 —

1. 문제의 정체
   - 세션이 컨테이너 메모리 안에 있다
   - 컨테이너 재시작·추가 시 세션이 따라가지 못한다
   - 사용자는 새로고침 한 번에 로그아웃된다
2. 해결 방향
   - 세션을 **컨테이너 바깥**에 두면 된다
   - 모든 컨테이너가 **같은 곳을 바라보면** 된다
   - 포스트잇이 아니라 **공용 게시판** 같은 것
3. 저장소의 조건
   - 속도가 빨라야 한다 (요청마다 읽는다)
   - 컨테이너가 죽어도 살아 있어야 한다

공용 게시판.

답이 보였습니다. 각자의 서랍을 없애고 사무실 한가운데에 게시판 하나를 걸어 두는 겁니다. 포스트잇은 전부 그 게시판에 붙입니다. 누가 앉든, 자리를 옮기든, 게시판만 보면 됐습니다.

3.2 두 대로 늘린 순간, 세션이 사라진다

말로만 하면 감이 오지 않으니, 두 대를 직접 띄워 보겠습니다. 먼저 `docker-session-share` 레포로 이동합니다.

터미널 레포 이동

```
cd docker-session-share
```

이 레포는 **이미 Redis 설정이 들어간 최종 구성**이지만, 지금은 **Redis를 일부러 끈 상태**로 한 번 돌려 보겠습니다. 인메모리 세션의 한계를 눈으로 확인하는 것이 목적입니다.

`app1/src/main/resources/application.properties`를 열고 세 줄을 잠깐 주석 처리합니다.

실습 1 app1/application.properties. Redis 연결 임시 비활성화

```
# spring.session.store-type=redis
# spring.data.redis.host=${SPRING_REDIS_HOST:redis}
# spring.data.redis.port=${SPRING_REDIS_PORT:6379}
```

`app2/src/main/resources/application.properties`도 같은 세 줄을 주석 처리합니다.

이 상태로 띄우면 `spring.session.store-type`이 없으니 Spring Session은 Redis를 쓰지 않습니다. 기본값인 Tomcat 인메모리 세션으로 돌아갑니다. 즉 두 Spring 컨테이너가 **각자의 메모리에만** 세션을 저장합니다.

`docker-compose.yml`의 Redis 서비스 블록만 잠깐 주석 처리한 뒤 띄웁니다.

터미널 실험 3-1 실행. 인메모리 세션으로 두 대 띄우기

```
docker compose up --build
```

빌드가 끝나고 네 컨테이너가 뜹니다. 그중 Redis는 주석 처리했으니 실제로는 Nginx·app1·app2 세 컨테이너만 동작합니다.

[CAPTURE NEEDED: docker compose up --build 실행 후 nginx, app1, app2 세 컨테이너가 Running으로 뜬 Docker Desktop 화면. redis 컨테이너는 없음]

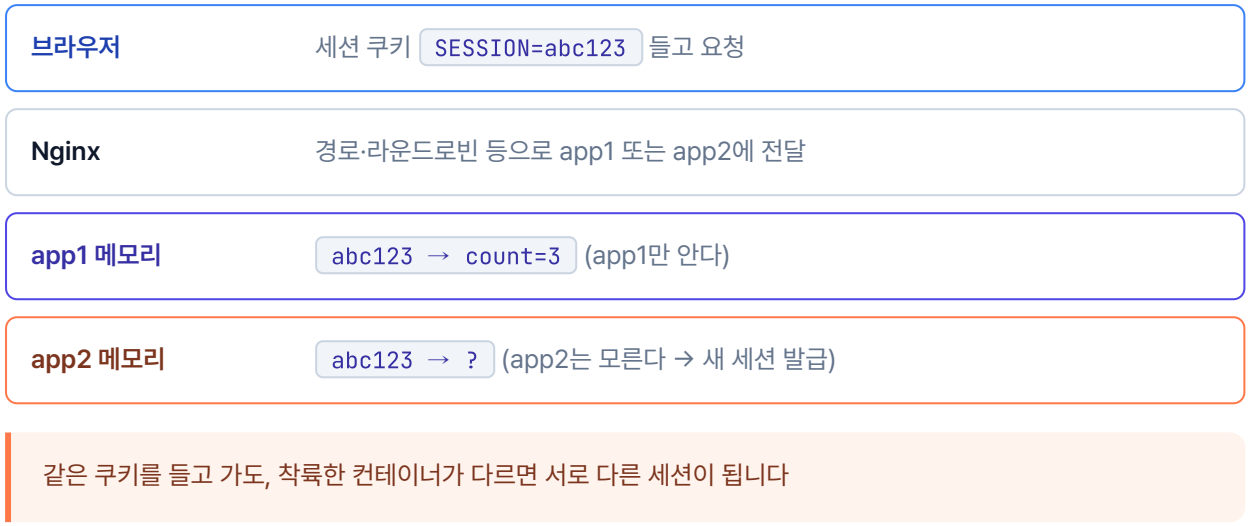
브라우저를 열고 `http://localhost/app1/`에 여러 번 접속합니다. 그다음 `http://localhost/app2/`로 이동합니다. 화면에는 서버 이름과 방문 횟수가 표시됩니다.

[CAPTURE NEEDED: 브라우저에서 http://localhost/app1/ 첫 접속. "app1" 서버 이름과 "현재 세션 방문 횟수: 1"]

[CAPTURE NEEDED: http://localhost/app2/로 이동했을 때. "app2" 서버 이름과 "현재 세션 방문 횟수: 1" (2가 아니라 1로 리셋됨)]

방문 횟수가 `/app1`에서 `1 → 2 → 3`으로 올라가다가, `/app2`로 넘어가는 순간 다시 `1`로 리셋됩니다. `/app1`로 돌아오면 원래 값이 아니라 또 `1`이 될 수도 있습니다. Nginx가 매번 어느 쪽으로 보낼지 정하는데, 두 컨테이너가 서로의 세션을 모르기 때문입니다.

그림 3-1. 인메모리 세션 구조 (세션이 컨테이너 안에 갇힘)



그래서 동료 새로고침이 풀렸구나.

여기서 한 번 시도해 볼 수 있는 임시 봉합이 있습니다. Nginx에 **Sticky Session** 설정을 붙이는 겁니다. "같은 브라우저에서 온 요청은 항상 같은 컨테이너로 보내라"는 지시입니다. Nginx는 보통 IP 해시나 쿠키 해시로 이걸 흉내 냅니다. 이 레포의 `nginx.conf`에는 없지만, `upstream` 블록 안에 `ip_hash;`를 한 줄 넣으면 됩니다.

참고 Sticky Session 임시 봉합 (권장하지 않음)

```
upstream app_upstream {
    ip_hash;                # 같은 IP는 항상 같은 서버로
    server app1:8080;
    server app2:8080;
}
```

해 보면 **증상은 잠깐 사라집니다.** 새로고침해도 로그아웃되지 않습니다. 하지만 문제가 세 가지 더 생깁니다.

그림 3-2. Sticky Session이 해결하지 못하는 것

컨테이너 하나가 죽으면	거기에 묶인 사용자 전부 로그아웃. 앞에서 라우팅을 흉내 냈을 뿐, 세션은 여전히 그 안에만 있음
부하가 쏠린다	큰 고객이 계속 같은 컨테이너로만 가면 그 컨테이너만 뜨거워진다
배포가 어렵다	롤링 업데이트로 컨테이너를 차례로 내렸다 올리는 순간 매번 세션이 끊긴다

> 증상을 가렸을 뿐, 원인(세션이 컨테이너 안에 있음)은 그대로입니다

원인은 세션이 어디 있느냐였다.

Sticky Session은 "사용자를 세션 쪽으로 데려가는" 해법이었습니다. 반대로 가야 했습니다. **세션을 사용자 쪽이 아니라, 모든 컨테이너가 보는 공용 장소로** 옮기는 쪽. 그게 다음 절의 주제입니다.

3.3 공용 게시판 하나를 사무실 한가운데에

세션을 어디에 둬야 할지, 조건부터 정리하면 세 가지였습니다.

첫째, **빠르다.** HTTP 요청이 들어올 때마다 세션을 조회하니 DB에 넣으면 너무 느립니다. 메모리 수준의 속도가 필요합니다.

둘째, **컨테이너가 죽어도 살아 있다.** Spring 컨테이너와 독립된 자기 프로세스·자기 저장소를 가져야 합니다.

셋째, **키 하나로 가져가고 넣을 수 있다.** 세션은 본질적으로 `세션ID → 데이터` 구조입니다. 복잡한 쿼리가 필요 없습니다. 키 하나로 읽고 씁니다.

이 셋을 모두 만족하는 것 중에 **Redis**가 있었습니다. 인메모리 키-값 저장소고, 독립 프로세스로 돌아가고, `SET` / `GET` 한 번에 끝납니다. 거기에 **TTL**(Time To Live)까지 기본으로 달려 있어서 "30분 뒤에 자동으로 사라져라"를 한 줄로 설정할 수 있었습니다.

[GEMINI PROMPT: 사무실 한가운데에 큰 공용 게시판(Redis)이 걸려 있고, 그 주변에 책상 두 개(Spring app1, Spring app2)가 있다. 각 책상에서 손을 뻗어 게시판에 포스트잇(세션 데이터)을 붙이고 있는 모습. 포스트잇에는 "SESSION:abc123"처럼 키가 적혀 있다. 사람들이 어느 책상에 앉든 같은 게시판을 본다. 깔끔한 일러스트, 흰 배경, 인디고·오렌지 악센트.]

그림 3-3. 각자의 서랍이 아니라 하나의 게시판을 본다

구조를 잡는 건 어렵지 않았습니다. Spring Boot 쪽은 **Spring Session Data Redis** 라는 스타터가 이미 있었습니다. `application.properties`에 한 줄만 쓰면 `HttpSession` 인터페이스를 그대로 두면서 저장소만 Redis로 바뀝니다. 컨트롤러의 `session.setAttribute()` 코드는 한 글자도 고치지 않아도 됐습니다.

Nginx는 이 그림에서 **게시판을 바라보지 않습니다.** 요청을 받아서 두 Spring 컨테이너 중 한 쪽으로 던져 주기만 합니다. 세션을 공유하는 건 Spring들 쪽이지 Nginx가 아니라는 점이 중요했습니다. Sticky Session 같은 걸 쓰지 않아도 된다는 뜻이기도 했습니다.

그림 3-4. 두 Spring 서버가 같은 Redis를 바라보는 구조

이제 이 그림을 그대로 코드로 옮기겠습니다.

3.4 Spring Session Data Redis 붙이기

먼저 **Spring 쪽**부터 손봅니다. 두 서버는 로직이 같으니 app1에만 설명하고, app2에도 동일하게 적용합니다.

3.4.1 의존성 추가

`app1/build.gradle`의 `dependencies` 블록에 두 줄을 추가합니다.

실습 2 app1/build.gradle. Redis + Spring Session 의존성

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-mustache'

    // Redis에 접속할 수 있는 Spring Data 모듈
    implementation 'org.springframework.boot:spring-boot-starter-data-redis'

    // HttpSession을 Redis에 자동으로 저장해 주는 모듈
    implementation 'org.springframework.session:spring-session-data-redis'
}
```

두 줄을 나눈 이유가 있습니다. `spring-boot-starter-data-redis`는 **"Redis에 어떻게 접속하는가"** (호스트·포트·커넥션 풀·직렬화 등 연결 계층)를 담당합니다. `spring-session-data-redis`는 그 위에 얹혀서 **"HttpSession을 Redis에 어떻게 넣고 꺼내는가"** (세션 객체를 Redis 해시로 매핑, TTL 설정, 만료 이벤트 처리)를 담당합니다. Spring Session은 Redis 자체를 모르고 Spring Data의 커넥션만 빌려 씁니다.

의존성	담당
`spring-boot-starter-data-redis`	Redis 커넥션 (Lettuce 드라이버 · 직렬화 기본 설정)
`spring-session-data-redis`	`HttpSession` → Redis 해시 매핑 · 세션 TTL · 만료 이벤트

3.4.2 application.properties

그다음 `app1/src/main/resources/application.properties`에서, 3.2절에서 주석 처리했던 세 줄의 주석을 풉니다.

실습 3 app1/application.properties. Spring Session 저장소를 Redis로

```
# 서버 포트 (nginx 뒤에서 내부적으로만 씀)
server.port=8080

# 이 서버의 표시 이름 (HomeController에서 응답 HTML에 찍음)
server.name=app1

# Spring Session이 세션을 Redis에 저장하도록 지정
spring.session.store-type=redis

# Redis 호스트 이름. 환경변수(SPRING_REDIS_HOST)가 있으면 그 값, 없으면 "redis"
# "redis"는 docker-compose가 만드는 서비스명이자 컨테이너끼리의 호스트명
spring.data.redis.host=${SPRING_REDIS_HOST:redis}

# Redis 포트. 환경변수(SPRING_REDIS_PORT)가 있으면 그 값, 없으면 6379
spring.data.redis.port=${SPRING_REDIS_PORT:6379}
```

세 줄만 더 있으면 됩니다. `spring.session.store-type=redis` 한 줄이 "이제부터 세션을 Redis에 둡니다"라는 선언이고, 아래 두 줄은 **어디의 Redis**인지를 알려 주는 접속 정보입니다.

`spring.data.redis.host=redis`에서 `redis`가 IP 주소가 아니라는 점이 이상하게 보일 수 있습니다. 이건 **docker-compose가 만들어 주는 내부 DNS 이름**입니다. `docker-compose.yml`에서 서비스를 `redis:` 이름으로 정의하면, 같은 네트워크에 있는 다른 컨테이너는 `redis`라는 호스트명만으로 그 컨테이너에 접속할 수 있습니다. 3.5절에서 compose 파일을 만들면 이 구조가 드러납니다.

속성	값	설명
`spring.session.store-type`	`redis`	세션 저장소를 Tomcat 메모리에서 Redis로 전환
`spring.data.redis.host`	`redis` (compose 서비스명)	컨테이너 네트워크 안에서의 Redis 호스트명
`spring.data.redis.port`	`6379`	Redis 기본 포트

`app2/src/main/resources/application.properties`에도 같은 다섯 줄을 넣습니다. 차이는 `server.name=app2` 한 줄뿐입니다.

3.4.3 세션을 읽고 쓰는 코드

세션 공유가 실제로 동작하는지 눈으로 확인하려면, 요청이 올 때마다 세션에서 값을 읽어 **화면에 찍는 컨트롤러**가 필요합니다. 카카오 로그인의 `sessionUser`를 쓰면 가장 자연스럽지만, 이 챕터는 "세션 공유" 그 자체를 검증하는 것이 목적입니다. 그래서 단순하게 **방문 횟수(count)** 를 세션에 넣고 서버 이름과 함께 찍어 주는 컨트롤러 하나를 확인하겠습니다.

`app1/src/main/java/com/metacoding/spring_session_share_app1/HomeController.java`를 엽니다. 이 레포에는 이미 컨트롤러가 채워져 있습니다. 코드를 고치지 말고 읽어만 보겠습니다.

실습 4 HomeController.java. 세션 count를 읽고 +1 저장

```
@GetMapping("/")
public String home(HttpServletRequest request, Model model) {
    // 1. 요청에 실린 SESSION 쿠키로 HttpSession을 꺼낸다
    //    Spring Session이 뒤에서 Redis에 SELECT한다
    HttpSession session = request.getSession();

    // 2. 세션에서 count 값을 읽고, 없으면 1 · 있으면 +1
    Integer count = (Integer) session.getAttribute("count");
```

```
        count = (count == null) ? 1 : count + 1;

        // 3. 올린 값을 다시 세션에 저장
        //    Spring Session이 뒤에서 Redis에 UPDATE한다
        session.setAttribute("count", count);

        // 4. 뷰에 서버 이름과 count를 전달
        model.addAttribute("server", serverName);
        model.addAttribute("count", count);
        return "index";
    }
```

이 네 단계에서 코드로 보이는 건 여전히 Java의 `HttpSession` 입니다. `request.getSession()` 과 `session.setAttribute()` 뿐입니다. Redis를 직접 다루는 `redisTemplate.opsForHash()` 같은 코드가 **한 줄도 없습니다.** 그런데도 Spring Session이 중간에서 자동으로 Redis와 대화합니다.

호출	겉으로 하는 일	속으로 일어나는 일
`request.getSession()`	세션 가져오기	SESSION 쿠키 값으로 Redis 해시 `spring:session:sessions:{id}` 조회
`session.getAttribute("count")`	속성 꺼내기	조회해 둔 해시에서 필드 `sessionAttr:count` 읽기
`session.setAttribute("count", n)`	속성 저장	Redis 해시에 `HSET` + TTL 갱신

`server.name` 은 `application.properties` 의 `server.name=app1` 값이 `@Value` 로 주입됩니다. 이 값이 응답에 그대로 박히기 때문에, 브라우저에서 화면만 봐도 "방금 내 요청이 어느 컨테이너에 들어갔는가"를 알 수 있습니다.

뷰는 간단합니다. `app1/src/main/resources/templates/index.mustache` 만 확인하고 넘어갑니다.

참고 app1/src/main/resources/templates/index.mustache

```
<h1>{{server}}</h1>
<p>현재 세션 방문 횟수: {{count}}</p>
```

3.5 네 컨테이너를 한 배치도에

Spring 설정이 끝났으니 이제 **네 컨테이너를 한** `docker-compose.yml` 에 올립니다. 챕터 1에서는 app 한 덩어리였던 배치도가 여기서 크게 늘어납니다.

그림 3-5. docker-compose 네 서비스와 역할

nginx	호스트 80 → `/app1` · `/app2` 경로로 app1/app2에 프록시
app1	Spring Boot 서버 A. 내부 8080. Redis를 `redis:6379` 로 접속
app2	Spring Boot 서버 B. 코드 동일 · `server.name=app2` 만 다름
redis	`redis:7-alpine` . 6379 포트. 세션 데이터가 머무는 공용 게시판

> 네 컨테이너는 같은 기본 네트워크에 있어서 서비스명(redis·app1·app2)으로 서로를 부릅니다

3.5.1 docker-compose.yml

`docker-compose.yml` 을 열어 확인합니다.

실습 5 docker-compose.yml. 네 서비스 배치도

```
services:
  nginx:
    image: nginx:alpine
    container_name: session-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - app1
```

```
  app1:
    build:
      context: ./app1
    container_name: session-app1
    environment:
      - SPRING_REDIS_HOST=redis
      - SPRING_REDIS_PORT=6379
    depends_on:
      - redis

  app2:
    build:
      context: ./app2
    container_name: session-app2
    environment:
      - SPRING_REDIS_HOST=redis
      - SPRING_REDIS_PORT=6379
    depends_on:
      - redis

  redis:
    image: redis:7-alpine
    container_name: session-redis
    ports:
      - "6379:6379"
```

포인트 세 개만 짚겠습니다.

첫째, **Nginx만 호스트 포트를 노출**합니다. `ports: "80:80"` 줄은 nginx 서비스에만 있습니다. app1·app2에는 `ports`가 없습니다. 바깥에서 Spring에 직접 접근하지 못합니다. 모든 트래픽은 Nginx를 거쳐야 합니다. Redis는 `6379:6379`로 노출되어 있는데, 이건 3.7절에서 호스트 터미널에서 `redis-cli`로 들여다볼 수 있게 열어 둔 것입니다. 운영에서는 닫습니다.

둘째, `depends_on`**으로 기동 순서를 잡습니다.** `app1`·`app2`는 `redis`가 먼저 떠야 하고, `nginx`는 앞단이므로 두 app 컨테이너가 먼저 떠야 합니다. `depends_on`은 "먼저 컨테이너를 시작하라"는 지시일 뿐 "완전히 준비되기를 기다려라"는 아닙니다. 실무에서는 헬스체크를 함께 씁니다.

셋째, **환경변수로 Redis 주소를 주입**합니다. app1·app2의 `SPRING_REDIS_HOST=redis`가 아까 `application.properties`의 `${SPRING_REDIS_HOST:redis}`에서 참조되는 값입니다. docker-compose 네트워크에서는 서비스명 `redis`가 곧 호스트명이 되므로 이 값이 그대로 연결 대상이 됩니다.

> **이 책은 간결함을 우선한다**
> 운영 환경의 compose 파일에는 보통 **헬스체크**(`healthcheck:` 블록), **네트워크 분리**(`networks:` 명시 선언), **볼륨**(`volumes:`로 Redis 영속성 확보), **리소스 제한**(`deploy.resources`)이 더 붙습니다. 이번 챕터는 세션 공유의 본질에만 집중하기 위해 기본 네트워크와 임시 스토리지를 씁니다. 운영에서는 Redis에 `volumes: - redis-data:/data`를 붙여 영속성을 확보합니다.

3.5.2 nginx.conf

`nginx/nginx.conf`는 두 upstream을 만들고 경로로 분기합니다.

실습 6 nginx/nginx.conf. Path 기반 라우팅

```
events {
    worker_connections 1024;
}

http {
    # 분기 대상 서버 두 개
    upstream app1_upstream { server app1:8080; }
    upstream app2_upstream { server app2:8080; }

    server {
        listen 80;

        # /app1/* 요청은 app1으로
        location /app1 { proxy_pass http://app1_upstream/; }

        # /app2/* 요청은 app2로
        location /app2 { proxy_pass http://app2_upstream/; }

        # 기본 진입은 /app1/로 보냄
        location = / { return 302 /app1/; }

        # 프록시 헤더 표준 설정
        proxy_redirect off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

`upstream` 블록은 "이 이름으로 오는 요청을 어느 서버에 보낼지" 매핑하는 자리입니다. `proxy_pass http://app1_upstream/;` 끝의 슬래시(`/`)는 "Nginx가 받은 경로에서 `/app1` 부분은 떼고 넘겨라"는 의미입니다. 그래서 브라우저가 `/app1/something`으로 왔을 때, Spring 서버는

`/something`만 받아서 처리합니다. Spring의 `HomeController`는 `@GetMapping("/")`만 있으므로 `/app1/`로 오면 루트로 떨어집니다.

여기서 Sticky Session이 필요 없습니다. `ip_hash;`도, 쿠키 해시도 없습니다. 세션은 어차피 Redis에 있기 때문에 Nginx가 어느 쪽으로 보내든 상관없습니다.

3.6 같은 세션으로 두 서버 왕복하기

구성이 끝났으니 실행합니다. 3.2절에서 주석 처리했던 `application.properties`의 세 줄을 다시 살려 둔 상태여야 합니다.

터미널 실험 3-2 실행. 네 컨테이너 기동

```
docker compose up --build
```

첫 빌드는 Gradle 빌드가 두 번(app1·app2) 돌기 때문에 몇 분 걸립니다. 마지막에 Nginx·app1·app2·Redis 네 줄이 차례로 `Started`·`Ready` 로그를 찍으면 준비 완료입니다.

[CAPTURE NEEDED: docker compose up --build 실행 후 Docker Desktop Containers 탭에 session-nginx, session-app1, session-app2, session-redis 네 컨테이너가 모두 Running 상태]

브라우저 주소창에 `http://localhost/app1/`을 입력합니다. 화면에 서버 이름과 방문 횟수가 뜹니다.

[CAPTURE NEEDED: http://localhost/app1/ 접속 결과. 큰 글씨로 "app1", 아래에 "현재 세션 방문 횟수: 1"]

주소창을 `http://localhost/app2/`로 바꿔 엔터를 칩니다. 서버 이름이 `app2`로 바뀌었지만, 방문 횟수는 `2`로 올라갑니다. 한 번 더 app1으로 돌아오면 `3`이 됩니다.

[CAPTURE NEEDED: http://localhost/app2/ 이동 결과. "app2" 서버 이름에 방문 횟수 "2"]

[CAPTURE NEEDED: 다시 http://localhost/app1/로 돌아온 결과. "app1" 서버 이름에 방문 횟수 "3"]

그림 3-6. 세션 공유 검증. 서버 이름은 바뀌지만 count는 이어진다

/app1 첫 접속	`server=app1` · `count=1` (세션 생성 · Redis에 기록)
/app2로 이동	`server=app2` · `count=2` (같은 세션ID로 Redis 조회)
/app1로 복귀	`server=app1` · `count=3` (끊김 없음)

> 서버 이름 = 이번 요청이 착륙한 컨테이너. count 연속성 = 세션이 공유되고 있다는 증거

더 확실하게 확인하려면 `docker compose restart session-app1` 한 번으로 app1만 껐다 켭니다. 재시작 뒤에도 count는 이어집니다. 이전 챕터의 Tomcat 인메모리 세션이었다면 `session-app1`의 메모리가 지워져서 count가 사라졌을 겁니다.

서랍을 엎어도 게시판은 그대로였다.

3.7 Redis 안을 들여다보기

count가 이어지는 건 눈으로 봤지만, **정말 Redis 안에 세션이 들어가 있는가**는 한 단계 더 확인해 볼 수 있습니다. 새 터미널에서 Redis 컨테이너에 들어갑니다.

터미널 실험 3-3 실행. Redis CLI 접속

```
docker exec -it session-redis redis-cli
```

`redis-cli` 프롬프트가 뜨면 `KEYS *`로 지금 저장된 키를 전부 훑어보겠습니다.

터미널 Redis 키 조회

```
127.0.0.1:6379> KEYS *
```

Spring Session이 만들어 둔 키가 세 종류로 나옵니다.

[CAPTURE NEEDED: redis-cli에서 KEYS * 결과. spring:session:sessions:{uuid}, spring:session:expirations:{timestamp}, spring:session:sessions:expires:{uuid} 세 종류 키가 찍힌 화면]

키 패턴	역할
`spring:session:sessions:{id}`	세션 본체. 해시(Hash) 타입. 속성·생성 시각·만료 시각이 필드로 들어간다
`spring:session:sessions:expires:{id}`	개별 세션의 만료 타이머. Redis의 Expire 이벤트가 여기를 본다
`spring:session:expirations:{timestamp}`	분 단위로 묶은 만료 예약 집합. Spring Session이 주기적으로 스캔

세션 본체가 정말 해시 구조인지 하나 골라서 열어 보겠습니다.

터미널 세션 해시 조회

```
127.0.0.1:6379> HGETALL spring:session:sessions:{위에서 본 id}
```

필드 다섯 개쯤이 줄줄이 나옵니다.

[CAPTURE NEEDED: HGETALL 결과. creationTime, lastAccessedTime, maxInactiveInterval, sessionAttr:count (값이 3 같은 숫자), sessionAttr:sessionUser 같은 필드가 찍힌 화면]

필드	값	의미
`creationTime`	긴 숫자(에폭 밀리초)	세션이 처음 만들어진 시각
`lastAccessedTime`	긴 숫자	마지막으로 이 세션에 접근한 시각. 매 요청마다 갱신
`maxInactiveInterval`	`1800`	비활성 만료 시간(초). 기본 30분
`sessionAttr:count`	직렬화된 바이트	`session.setAttribute("count", 3)`이 들어간 자리

`sessionAttr:{키}` 필드에 들어간 값이 **진짜 내가 저장한 데이터**입니다. `sessionAttr:count`는 단순 Integer라 알아볼 만하지만, 챕터 2에서 넣은 `sessionAttr:sessionUser` 같은 복잡한 객체는 바이트 덩어리로 보입니다. 이 값을 Redis가 그대로 저장하고 꺼내려면 직렬화(serialize)와 역직렬화

(deserialize)가 필요합니다. Spring Session의 기본은 **JDK 직렬화**입니다. 이 때문에 세션에 넣을 객체는 `Serializable`을 구현해야 합니다.

직렬화 실험에서 생각해 볼 것들

- **JDK 직렬화의 한계**: 바이트가 불투명해서 `redis-cli`로 열어도 사람이 읽기 어렵습니다. 두 서버의 클래스 버전이 어긋나면 `InvalidClassException`이 납니다. 운영에서는 **JSON 직렬화**(`GenericJackson2JsonRedisSerializer`)로 바꿔 두는 편이 디버깅과 버전 관리에 유리합니다. 이 스위치는 `RedisSerializer`를 빈으로 등록하여 `spring.session.redis.flush-mode`와 함께 조정합니다
- **TTL은 요청마다 갱신됩니다**: `maxInactiveInterval`은 "마지막 요청 후 몇 초까지 유지"입니다. 사용자가 계속 요청하는 동안은 만료되지 않고, 손을 놓으면 정확히 그 시간이 지난 뒤 Redis에서 사라집니다. Redis의 `EXPIRE`가 이 일을 대신합니다
- **세션과 JWT는 다른 도구**: 챕터 2의 OIDC REST 버전에서 만든 `JwtUtil.create`는 토큰 한 장에 정보를 담는 무상태 방식입니다. 이번 챕터의 Redis 세션은 서버가 상태를 들고 있는 유상태 방식입니다. 어느 쪽을 쓸지는 보통 **로그아웃 처리 · 토큰 강제 무효화 요구**가 있느냐로 갈립니다. 세션은 Redis에서 키 하나만 지우면 즉시 로그아웃, JWT는 별도의 블랙리스트가 필요합니다
- **Redis가 죽으면 세션이 전부 날아갑니다**: 인메모리가 빠른 대가입니다. 운영에서는 `appendonly yes`(AOF)나 `save` 설정으로 디스크에도 기록해 두고, 더 중요한 서비스에서는 Redis Sentinel 또는 Cluster로 다중화합니다. 이 책 범위는 여기까지만 다룹니다

redis-cli에서 나옵니다.

터미널 redis-cli 종료

```
127.0.0.1:6379> exit
```

그리고 네 컨테이너를 전부 내립니다.

터미널 실험 3-4 실행. 컨테이너 정리

```
docker compose down
```

`docker compose down`이 돌았습니다. 오픈이는 네 컨테이너가 차례로 Removed로 표시되는 걸 지켜봤습니다. 어제까지는 컨테이너 한 대짜리 서비스였는데, 오늘은 앞단 라우터와 뒤편 저장소까지 딸린 네 덩어리짜리 서비스가 되었습니다.

동료가 다시 의자를 돌렸습니다.

동료 : "이제 새로고침해도 로그아웃 안 되겠네요."

오픈이 : "네. 컨테이너 재시작해도 안 풀려요. 로그인 상태가 Redis에 있으니까요."

동료 : "그럼 다음엔 뭐예요? 이미지 업로드는 언제 해요? 프로필 사진 바꾸고 싶은데."

이미지.

오픈이는 살짝 눈썹을 올렸습니다. 프로필 사진을 서버에 올리는 건 처음 해 보는 일이었습니다. 로그인도 해결됐고 세션도 공유됐으니 이제 사용자가 데이터를 **실제로 남기는** 쪽으로 한 걸음 더 가야 했습니다.

근데 사진을 어디에 두지. Spring 컨테이너 안에 저장하면 아까처럼 재시작하면 날아갈 텐데.

수첩 귀퉁이에 다시 물음표가 붙었습니다.

사용자가 올린 파일을 어디에 담을까. 일단 작은 이미지부터.

다음 챕터의 문이 열리고 있었습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
책상 서랍	인메모리 세션 (Tomcat HttpSession)	서블릿 컨테이너 프로세스의 힙 메모리에 보관되는 세션. 프로세스 종료 시 소실
공용 게시판	Redis	인메모리 키-값 저장소. 독립 프로세스로 구동되며 TTL을 기본 제공

한 줄로 저장소 전환	Spring Session	`HttpSession` 인터페이스를 가린 채 저장소를 Redis·JDBC·MongoDB 등으로 교체 가능하게 해 주는 모듈
공용 게시판 붙이는 스타터	spring-session-data-redis	Spring Session의 Redis 구현체. `HttpSession` 속성을 Redis 해시로 매핑
포스트잇 자동 만료	TTL (Time To Live)	키가 저장된 뒤 일정 시간이 지나면 자동 삭제되는 속성. `maxInactiveInterval`이 세션 TTL
같은 브라우저는 같은 서버로	Sticky Session	로드 밸런서가 특정 클라이언트를 항상 같은 백엔드로 고정시키는 방식. 세션 공유의 대체가 아닌 임시 봉합
경로로 나누어 보내기	Nginx Path 기반 라우팅	`location /path`와 `upstream`을 조합해 요청을 다른 백엔드로 분기하는 Nginx 구성
docker-compose의 서비스 이름 호출	compose 네트워크 DNS	같은 compose 파일에 정의된 서비스는 서비스명을 호스트명처럼 사용해 서로 접속
세션을 바이트로 만들기	직렬화 (Serialization)	객체를 바이트 배열로 변환. Spring Session 기본은 JDK 직렬화. JSON으로 교체 가능
스프링 세션 저장 방 번호	`spring:session:sessions:{id}`	Spring Session이 세션 본체를 저장하는 Redis 해시 키 규칙

이것만은 기억하자

- **세션이 컨테이너 안에 있으면 두 대로 못 늘어난다.** 서버 메모리는 프로세스와 함께 사라지고, 옆 컨테이너와 공유되지 않는다
- **세션 저장소는 바깥으로 꺼낸다.** Redis 같은 공용 키-값 저장소에 두면 모든 컨테이너가 같은 세션을 본다

- **Spring Session은 코드에 손대지 않는다.** `HttpSession`을 그대로 쓰면서 저장소만 `spring.session.store-type=redis` 한 줄로 전환한다
- **Sticky Session은 응급 처치다.** 증상만 가리고 컨테이너 하나가 죽는 순간 다시 드러난다. 진짜 해법은 세션을 바깥으로 꺼내는 것
- **다음 챕터에서는** 사용자 프로필 이미지를 서버에 올리고 보여줍니다. 가장 단순한 방식인 **Base64 + DB 저장**부터 손으로 만들어 보고, 그 한계를 체감합니다 (Spring 이미지 업로드)